ABSTRACT

Embedded game development has gained significant attention as a practical approach to understanding real-time systems, hardware–software integration, and user interaction in resource-constrained environments. This project presents the design and implementation of an Arduino-based embedded game using an OLED display, inspired by the popular Dino Runner game. The proposed system integrates graphical rendering on a 128×64 OLED display, real-time score tracking using a TM1637 4-digit 7-segment display, user input through a push button, and audio feedback via a buzzer.

The system focuses on implementing core game mechanics such as character movement, obstacle generation, collision detection, and score computation using efficient embedded C/C++ programming techniques. Time-based logic is employed to manage gameplay flow and scoring, demonstrating real-time processing capabilities of the Arduino Uno platform. The developed game provides a compact, low-cost, and interactive embedded solution suitable for educational and experimental purposes. The project highlights the potential of microcontroller-based platforms for basic game development, human–machine interaction, and embedded system design.

Keywords

Arduino Uno, Embedded Systems, OLED Display, TM1637, Game Development

1. INTRODUCTION

Embedded systems have become an integral part of modern electronic devices, supporting a wide range of applications from consumer electronics to industrial automation. With the increasing demand for compact, low-cost, and interactive systems, microcontroller-based platforms such as Arduino have gained popularity in education, research, and prototyping. One effective way to understand embedded concepts such as real-time processing, peripheral interfacing, and hardware–software co-design is through embedded game development.

Games implemented on embedded platforms provide an interactive environment to explore system timing, input handling, graphical display control, and efficient memory utilization under hardware constraints. Unlike high-level gaming platforms, embedded games must operate with limited processing power, memory, and display resolution, making design optimization a critical aspect.

This project focuses on the development of an embedded Dino Runner game using the Arduino Uno and an OLED display. By integrating graphical rendering, user input through a push button, score visualization using a 4-digit 7-segment display, and sound feedback via a buzzer, the system demonstrates practical embedded design techniques. The proposed implementation serves as an educational model for understanding real-time embedded programming, human–machine interaction, and the application of microcontrollers in interactive system development.

2. OBJECTIVES

The main objectives of this project are:

- To design and develop an embedded game using the Arduino Uno microcontroller
- To interface an OLED display for real-time graphical output
- To implement character movement, obstacle generation, and collision detection logic
- To display the player's score using a TM1637 4-digit 7-segment display
- To integrate a push button for user interaction and game control
- To generate audio feedback using a buzzer for in-game events
- To understand real-time embedded programming and hardware–software integration.

3. SYSTEM OVERVIEW

The proposed embedded game development system is designed to implement an interactive and real-time gaming application using a microcontroller-based platform. The system integrates an Arduino Uno microcontroller, an OLED display, user input components, and output peripherals to create a compact and responsive game environment. The primary objective of the system is to demonstrate embedded system concepts such as real-time processing, peripheral interfacing, and hardware–software integration through game-based implementation.

The system utilizes a 128×64 OLED display to render game graphics, including the player character, obstacles, score information, and game status messages. User interaction is facilitated through a push button that allows the player to control game actions such as starting the game and performing jumps. A TM1637-based 4-digit 7-segment display is incorporated to continuously display the player's score, providing clear and independent numerical feedback. Additionally, a buzzer is used to generate audio signals for gameplay events such as jumping, score updates, and game-over conditions.

For game execution, the Arduino Uno processes input signals, updates game logic, performs collision detection, and controls display refresh operations in real time. Time-based programming techniques are employed to manage obstacle movement, scoring, and gameplay speed. The coordinated operation of all hardware and software components ensures smooth gameplay and responsive control. Overall, the system provides an effective platform for understanding embedded game development and real-time system behavior in resource-constrained environments.

4. BLOCK DIAGRAM

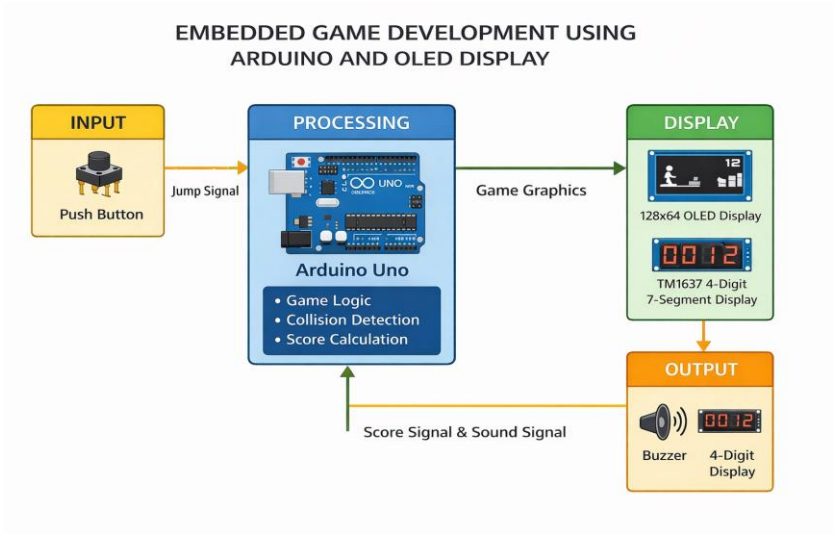

Figure 1: Block diagram

5. HARDWARE REQUIREMENTS

- Personal Computer / Laptop
- Arduino Uno Microcontroller
- OLED Display (128×64, SH1106 / SSD1306)
- TM1637 4-Digit 7-Segment Display
- Push Button
- Buzzer
- Breadboard
- Connecting Wires
- USB Cable for Arduino Programming

6. SOFTWARE REQUIREMENTS

- Arduino IDE
- Embedded C / C++ Programming Language
- U8glib / Adafruit SSD1306 Library (for OLED display)
- TM1637Display Library
- USB Driver for Arduino Uno
- Windows / Linux Operating System

7. WORKING PRINCIPLE

The embedded game development system operates by integrating user input, real-time processing, graphical display, and output feedback within a microcontroller-based environment. The Arduino Uno continuously monitors input signals from the push button to control game actions such as starting the game and initiating character jumps. These input signals are processed in real time to ensure immediate system response.

The core game logic is executed within the Arduino, where time-based control techniques are used to manage character movement, obstacle generation, and score calculation. The system continuously updates the game state by checking for collision conditions between the player character and obstacles. Upon detecting a collision, the system triggers a game-over event.

Visual output is generated on the OLED display, where the player character, obstacles, score indicators, and game status messages are rendered dynamically. The player's score is simultaneously displayed on the TM1637 4-digit 7-segment display, providing clear numerical feedback. Audio signals are generated through a buzzer to indicate events such as jumping, score increments, and game termination.

Overall, the system operates in a continuous loop, enabling real-time interaction between the user and the game. The coordinated operation of input handling, processing, display updates, and sound generation demonstrates the principles of real-time embedded system design and interactive application development.

8. METHODOLOGY

The proposed embedded game development project is implemented using the Arduino Uno platform to demonstrate real-time interaction, display control, and hardware–software integration. The system is designed to simulate a simple Dino Runner game, where the player controls a character to avoid obstacles while the score increases over time.

1) Selection of Hardware Components

The required hardware components were selected to support real-time game execution and user interaction. The Arduino Uno microcontroller serves as the central processing unit. A 128×64 OLED display is used for graphical rendering of the game environment, while a TM1637-based 4-digit 7-segment display is used to display the player's score. A push button is used as the primary input device, and a buzzer provides audio feedback for gameplay events.

2) Circuit Design and Interfacing

All components were interfaced with the Arduino Uno using appropriate digital and I2C connections.

- The OLED display was connected using the I2C communication protocol for efficient data transfer.
- The TM1637 display was interfaced using two digital pins for clock and data signals.
- The push button was connected to a digital input pin with proper pull-up or pull-down configuration.
- The buzzer was connected to a digital output pin to generate sound alerts.

Proper power supply and grounding were ensured to maintain stable system operation.

3) Game Logic Development

The game logic was implemented using Embedded C/C++ programming in the Arduino IDE. The logic includes:

- Initialization of all hardware components
- Creation of the player character and obstacle objects
- Continuous obstacle movement across the screen
- Jump control logic based on push-button input
- Score calculation based on gameplay duration

A continuous loop structure ensures real-time execution and smooth gameplay.

4) Collision Detection and Score Handling

Collision detection logic was implemented by continuously comparing the position of the player character and obstacles on the OLED display. When a collision is detected, the game terminates and a game-over message is displayed. The player's score is updated in real time and displayed on the TM1637 7-segment display throughout the gameplay.

5) Audio Feedback Implementation

Audio feedback was generated using a buzzer to enhance user experience. Different sound signals were programmed for:

- Jump actions
- Score increment events
- Game-over condition

6) Display Rendering and Score Visualization

The OLED display continuously updates the graphical elements of the game, including the player character, obstacles, ground line, and game status messages. Efficient drawing techniques are used to refresh only the required screen areas, ensuring smooth animation despite limited memory and processing capability.

The TM1637-based 4-digit 7-segment display independently shows the player's score in real time. This separation of graphical and numerical displays improves readability and reduces processing overhead on the OLED display.

7) Game Over Handling and Restart Mechanism

When a collision between the player character and an obstacle is detected, the system immediately halts obstacle movement and displays a "Game Over" message on the OLED screen. A distinct audio signal is generated through the buzzer to indicate the end of the game.

The system then waits for a push-button input to reset all game parameters, including score, character position, and obstacle state, allowing the player to restart the game without resetting the hardware.

8) Real-Time System Execution

The entire system operates in a continuous execution loop, enabling real-time input detection, game state updates, and output rendering. Timing functions are used to control animation speed, obstacle movement, and score increment rate. This real-time operation demonstrates effective embedded system design principles such as event handling, timing control, and responsive interaction.

9) System Validation and Testing

The developed system was tested under multiple gameplay scenarios to verify correct input handling, display updates, collision detection accuracy, and sound generation. Stable performance and consistent response times confirmed the reliability of the embedded game implementation.

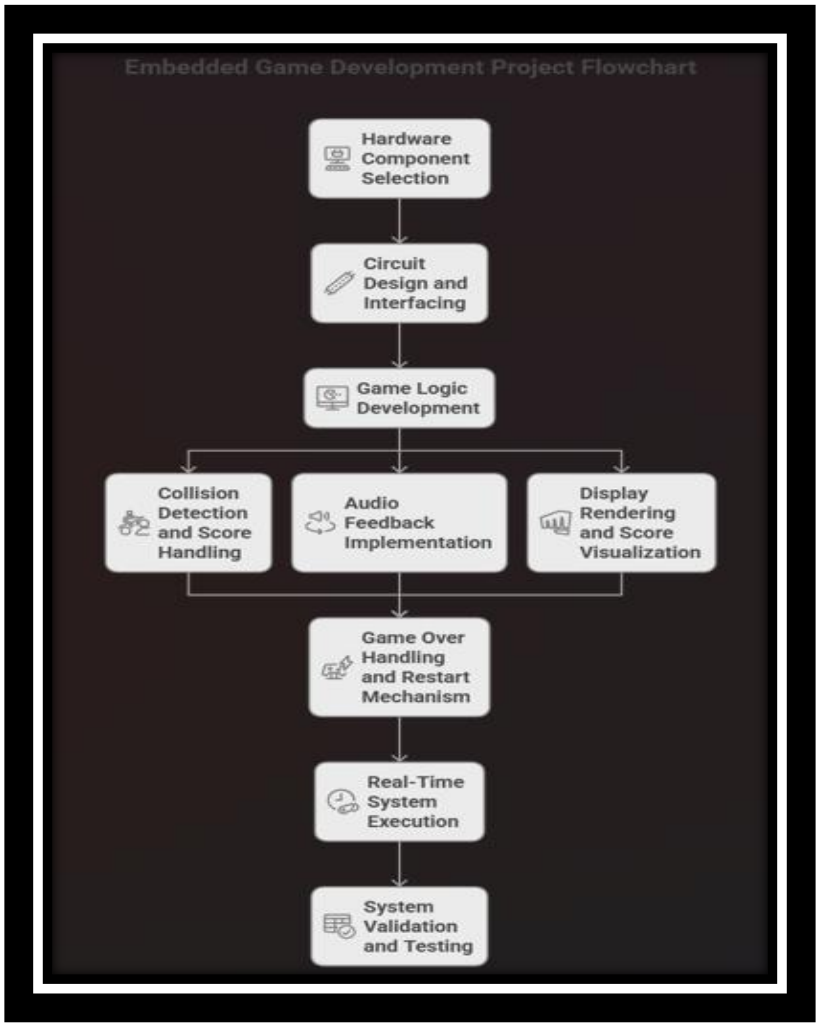

Figure 2: Methodology

APPENDEX

```
extern "C" {
  #include <stdlib.h>
  #include <string.h>
  #include <inttypes.h>
}
#include "TM1637Display.h"
#include <Arduino.h>
#define TM1637_I2C_COMM1    0x40
#define TM1637_I2C_COMM2    0xC0
#define TM1637_I2C_COMM3    0x80
//
//      A
//     ---
//  F |   | B
//     -G-
//  E |   | C
//     ---
//      D
const uint8_t digitToSegment[] = {
 // XGFEDCBA
  0b00111111,    // 0
  0b00000110,    // 1
  0b01011011,    // 2
  0b01001111,    // 3
  0b01100110,    // 4
  0b01101101,    // 5
  0b01111101,    // 6
  0b00000111,    // 7
  0b01111111,    // 8
  0b01101111,    // 9
  0b01110111,    // A
  0b01111100,    // b
  0b00111001,    // C
  0b01011110,    // d
  0b01111001,    // E
  0b01110001     // F
  };
```

```
static const uint8_t minusSegments = 0b01000000;

TM1637Display::TM1637Display(uint8_t pinClk, uint8_t pinDIO, unsigned int bitDelay)
{
  // Copy the pin numbers
  m_pinClk = pinClk;
  m_pinDIO = pinDIO;
  m_bitDelay = bitDelay;

  // Set the pin direction and default value.
  // Both pins are set as inputs, allowing the pull-up resistors to pull them up
    pinMode(m_pinClk, INPUT);
    pinMode(m_pinDIO,INPUT);
  digitalWrite(m_pinClk, LOW);
  digitalWrite(m_pinDIO, LOW);
}

void TM1637Display::setBrightness(uint8_t brightness, bool on)
{
  m_brightness = (brightness & 0x7) | (on? 0x08 : 0x00);
}

void TM1637Display::setSegments(const uint8_t segments[], uint8_t length, uint8_t pos)
{
    // Write COMM1
  start();
  writeByte(TM1637_I2C_COMM1);
  stop();

  // Write COMM2 + first digit address
  start();
  writeByte(TM1637_I2C_COMM2 + (pos & 0x03));

  // Write the data bytes
  for (uint8_t k=0; k < length; k++)
    writeByte(segments[k]);

  stop();
```

```
  // Write COMM3 + brightness
  start();
  writeByte(TM1637_I2C_COMM3 + (m_brightness & 0x0f));
  stop();
}

void TM1637Display::clear()
{
    uint8_t data[] = { 0, 0, 0, 0 };
  setSegments(data);
}

void TM1637Display::showNumberDec(int num, bool leading_zero, uint8_t length,
uint8_t pos)
{
  showNumberDecEx(num, 0, leading_zero, length, pos);
}

void TM1637Display::showNumberDecEx(int num, uint8_t dots, bool leading_zero,
                                    uint8_t length, uint8_t pos)
{
  showNumberBaseEx(num < 0? -10 : 10, num < 0? -num : num, dots, leading_zero, length,
pos);
}

void TM1637Display::showNumberHexEx(uint16_t num, uint8_t dots, bool leading_zero,
                                    uint8_t length, uint8_t pos)
{
  showNumberBaseEx(16, num, dots, leading_zero, length, pos);
}

void TM1637Display::showNumberBaseEx(int8_t base, uint16_t num, uint8_t dots, bool
leading_zero,
                                    uint8_t length, uint8_t pos)
{
    bool negative = false;
  if (base < 0) {
      base = -base;
    negative = true;
  }
```

```
uint8_t digits[4];

 if (num == 0 && !leading_zero) {
  // Singular case - take care separately
  for(uint8_t i = 0; i < (length-1); i++)
    digits[i] = 0;
  digits[length-1] = encodeDigit(0);
 }
 else {
  //uint8_t i = length-1;
  //if (negative) {
  //  // Negative number, show the minus sign
  //    digits[i] = minusSegments;
  //  i--;
  //}

  for(int i = length-1; i >= 0; --i)
  {
      uint8_t digit = num % base;

    if (digit == 0 && num == 0 && leading_zero == false)
        // Leading zero is blank
      digits[i] = 0;
    else
        digits[i] = encodeDigit(digit);

    if (digit == 0 && num == 0 && negative) {
        digits[i] = minusSegments;
      negative = false;
    }

    num /= base;
  }

  if(dots != 0)
  {
    showDots(dots, digits);
  }
  }
  setSegments(digits, length, pos);
```

```
}

void TM1637Display::bitDelay()
{
  delayMicroseconds(m_bitDelay);
}

void TM1637Display::start()
{
  pinMode(m_pinDIO, OUTPUT);
  bitDelay();
}

void TM1637Display::stop()
{
  pinMode(m_pinDIO, OUTPUT);
  bitDelay();
  pinMode(m_pinClk, INPUT);
  bitDelay();
  pinMode(m_pinDIO, INPUT);
  bitDelay();
}

bool TM1637Display::writeByte(uint8_t b)
{
  uint8_t data = b;

  // 8 Data Bits
  for(uint8_t i = 0; i < 8; i++) {
    // CLK low
    pinMode(m_pinClk, OUTPUT);
    bitDelay();

  // Set data bit
    if (data & 0x01)
      pinMode(m_pinDIO, INPUT);
    else
      pinMode(m_pinDIO, OUTPUT);

    bitDelay();
```

```
// CLK high
    pinMode(m_pinClk, INPUT);
    bitDelay();
    data = data >> 1;
  }

  // Wait for acknowledge
  // CLK to zero
  pinMode(m_pinClk, OUTPUT);
  pinMode(m_pinDIO, INPUT);
  bitDelay();

  // CLK to high
  pinMode(m_pinClk, INPUT);
  bitDelay();
  uint8_t ack = digitalRead(m_pinDIO);
  if (ack == 0)
    pinMode(m_pinDIO, OUTPUT);

  bitDelay();
  pinMode(m_pinClk, OUTPUT);
  bitDelay();

  return ack;
}

void TM1637Display::showDots(uint8_t dots, uint8_t* digits)
{
    for(int i = 0; i < 4; ++i)
    {
        digits[i] |= (dots & 0x80);
        dots <<= 1;
    }
}

uint8_t TM1637Display::encodeDigit(uint8_t digit)
{
  return digitToSegment[digit & 0x0f];
}
```

9. FLOW CHART

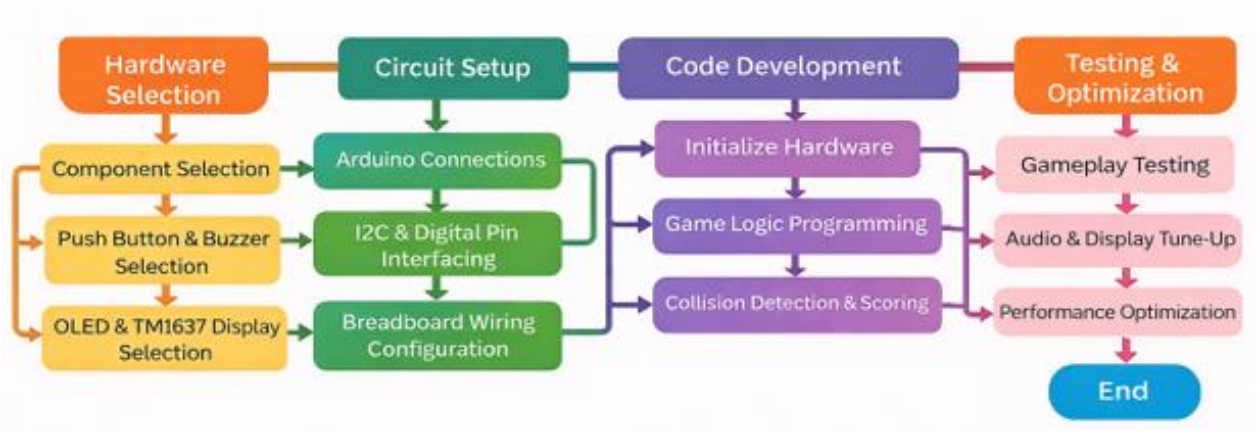

Figure 3: Architecture

Steps Involved in the System

i. Start

The embedded game system is initialized by powering on the Arduino Uno microcontroller.

ii. Hardware Initialization

All hardware components such as the OLED display, TM1637 7-segment display, push button, and buzzer are initialized and configured for proper operation.

iii. Game Environment Setup

The game screen layout is created on the OLED display, including the ground line, player character (Dino), and obstacle positions.

iv. User Input Detection

The push button input is continuously monitored to detect player actions such as jumping during gameplay.

v. Game Logic Execution

The game logic runs in a continuous loop where obstacles move across the screen, and player actions are processed in real time.

vi. Collision Detection

The system continuously checks for collisions between the player character and obstacles using position comparison logic.

vii. Score Calculation

The player's score is calculated based on gameplay duration and successful obstacle avoidance, and it is updated in real time.

viii. Score Display

The calculated score is displayed on the TM1637 4-digit 7-segment display throughout the gameplay.

ix. Audio Feedback Generation

The buzzer generates different sound effects for jump actions, score increments, and game-over events to enhance user experience.

x. Game Over Handling

When a collision is detected, the game stops, a game-over message is shown on the OLED display, and the final score is retained.

xi. End

The system ends the gameplay cycle, allowing the user to reset or restart the game.

10. ADVANTAGES

• **Hands-on Learning of Embedded Systems:**

The project provides practical experience in embedded system design, helping learners understand microcontroller programming, real-time execution, and hardware–software integration.

• **Real-Time Interaction and Control:**

The system supports real-time user interaction through a push button, demonstrating responsive input handling and time-critical processing.

• **Efficient Use of Limited Resources:**

The game is designed to operate within the memory, processing, and display constraints of the Arduino Uno, showcasing optimized embedded programming techniques.

• **Graphical Display Implementation:**

The use of a 128×64 OLED display enables basic graphical rendering, helping users understand low-resolution graphics handling in embedded environments.

• **Independent Score Visualization:**
The TM1637 4-digit 7-segment display provides clear and continuous score display without affecting the main game graphics.

• **Audio Feedback for Enhanced User Experience:**
The buzzer adds sound effects for gameplay events, improving user engagement and demonstrating multimedia integration in embedded systems.

• **Low-Cost and Simple Hardware Setup:**
The project uses affordable and easily available components, making it suitable for educational institutions and student projects.

• **Expandable and Customizable Design:**
The system can be enhanced with additional levels, controls, sensors, or displays, allowing further experimentation and learning.

11. APPLICATIONS

• **Educational Embedded Systems Learning:**
The project is useful for students to understand microcontroller programming, real-time processing, and hardware–software integration through a practical example.

• **Game-Based Learning Platforms:**
The embedded game demonstrates how simple games can be used as learning tools to teach logic development, timing control, and interactive system design.

• **Arduino and Microcontroller Training:**
It serves as a beginner-friendly project for Arduino training programs, workshops, and laboratory experiments.

• **Human–Machine Interface (HMI) Demonstration:**
The system illustrates basic HMI concepts using displays, buttons, and audio feedback in embedded environments.

• **Prototype Development for Embedded Graphics:**
The project can be used as a base model for developing graphical user interfaces and animations on low-resolution embedded displays.
• **Research and Mini Project Implementation:**
Suitable for academic mini-projects and demonstrations related to embedded systems, real-time systems, and IoT fundamentals.

12. RESULTS

The proposed embedded game development system was successfully implemented and tested using the Arduino Uno platform along with an OLED display, TM1637 7-segment display, push button, and buzzer. The system demonstrated stable real-time performance, responsive user interaction, and accurate display control throughout gameplay.

The graphical output on the 128×64 OLED display effectively rendered the Dino Runner game environment, including character movement and obstacle animation. The push button input was accurately detected, enabling smooth jump control without noticeable delay. Collision detection logic functioned correctly, reliably identifying collisions between the player character and obstacles and triggering the game-over condition.

The scoring mechanism performed consistently, with the TM1637 display updating the player's score in real time based on gameplay duration. Audio feedback generated through the buzzer successfully indicated key game events such as jumping, score progression, and game termination, enhancing user experience.

Overall testing confirmed that the system operates efficiently within the limited processing and memory resources of the Arduino Uno. The results validate the effectiveness of the proposed design in demonstrating embedded game development concepts, real-time processing, and hardware–software integration in a compact and low-cost embedded system.

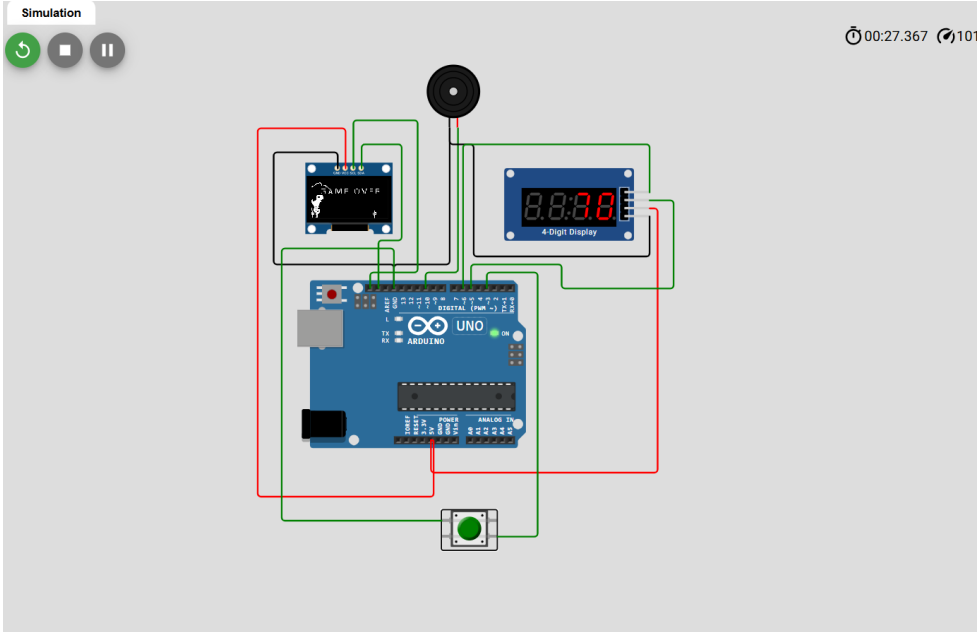

Figure 4: Arduino Uno Circuit Connections for OLED Game System

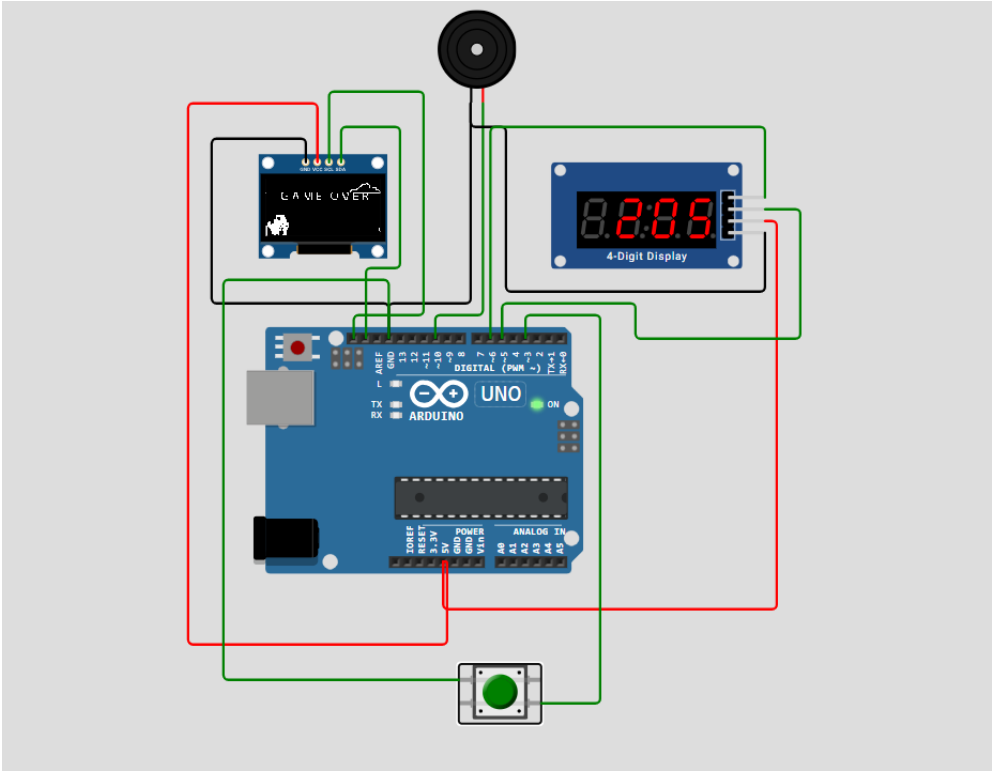

Figure 5: Real-Time Gameplay Output of Embedded Game System

i. FUTURE SCOPE

- **Addition of Multiple Game Levels:**
  Future versions of the game can include multiple difficulty levels with increasing obstacle speed and complexity.
- **Enhanced User Controls:**
  Additional push buttons or joystick modules can be integrated to allow more interactive and flexible player control.
- **High Score Storage:**
  Non-volatile memory such as EEPROM can be used to store and display high scores even after power reset.
- **Improved Graphics and Animation:**
  The system can be upgraded with smoother animations or a color OLED/TFT display for better visual experience.
- **Sound and Music Enhancement:**
  Advanced sound modules or tone generation techniques can be added to include background music and richer audio effects.
- **Multiplayer or Competitive Mode:**
  The project can be extended to support turn-based or score-comparison gameplay between multiple users.
- **Porting to Advanced Microcontrollers:**
  The game can be implemented on more powerful platforms like ESP32 or Raspberry Pi Pico for enhanced performance.
- **Wireless Connectivity:**
  Bluetooth or Wi-Fi modules can be integrated to enable wireless score sharing or remote gameplay monitoring.

13. CONCLUSION

This project successfully demonstrated the design and implementation of an embedded game development system using the Arduino Uno platform, effectively integrating graphical display control, real-time user interaction, audio feedback, and score management within the constraints of a microcontroller-based environment. By implementing a Dino Runner–style game, the system provided a practical demonstration of real-time embedded system operation and hardware–software co-design. The OLED display enabled efficient rendering of game graphics, while the TM1637 4-digit 7-segment display ensured accurate and continuous score visualization. Responsive player control was achieved through a push-button input mechanism, and audio cues generated by a buzzer enhanced user experience. Collision detection and timing-based score computation were implemented successfully, ensuring smooth and reliable gameplay execution. Overall, the project highlights the feasibility of developing interactive games on low-cost embedded platforms and serves as an effective learning model for embedded programming, peripheral interfacing, real-time processing, and system optimization, with scope for future enhancements such as improved graphics, additional controls, and advanced gameplay features.

REFERENCES

1. Arduino, *Arduino Uno Rev3 – Official Documentation*, Arduino.cc.
2. M. Margolis, *Arduino Cookbook*, 2nd ed., O'Reilly Media, 2012.
3. Adafruit Industries, *SSD1306 OLED Display Driver Documentation*.
4. U8glib / Adafruit, *GFX Library Documentation – Graphics Libraries for Arduino*.
5. TM1637 Display Datasheet, *4-Digit 7-Segment Display Module*.
6. Atmel Corporation, *ATmega328P Microcontroller Datasheet*.
7. S. Monk, *Programming Arduino: Getting Started with Sketches*, McGraw-Hill Education.
8. Arduino, *Arduino IDE – Integrated Development Environment User Guide*.
9. Simon Monk, *30 Arduino Projects for the Evil Genius*, McGraw-Hill Education.

10. John Boxall, *Arduino Workshop: A Hands-On Introduction with 65 Projects*, No Starch Press.
11. Microchip Technology Inc., *AVR Microcontroller Architecture and Programming Manual*.
12. Banzi, M., *Getting Started with Arduino*, O'Reilly Media.
13. D. Ibrahim, *Microcontroller Based Applied Digital Control*, John Wiley & Sons.
14. Raj Kamal, *Embedded Systems: Architecture, Programming and Design*, McGraw-Hill Education.
15. Frank Vahid and Tony Givargis, *Embedded System Design: A Unified Hardware/Software Introduction*, Wiley.
16. Michael Barr and Anthony Massa, *Programming Embedded Systems in C and C++*, O'Reilly Media.
17. Elektor International Media, *Arduino Projects Handbook*.
18. Jack Ganssle, *The Art of Designing Embedded Systems*, Newnes.
19. B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, Prentice Hall.
20. Adafruit Learning System, *Using Push Buttons and Buzzers with Arduino*.
21. Texas Instruments, *Introduction to Embedded System Design*.
22. IEEE Xplore Digital Library, *Research Articles on Embedded Systems and Microcontroller Applications*.